



ARA0110 PADRÕES DE PROJETO

Aula 08: Padrões de projeto comportamentais - parte 3





RECAPITULANDO





PADRÕES COMPORTAMENTAIS

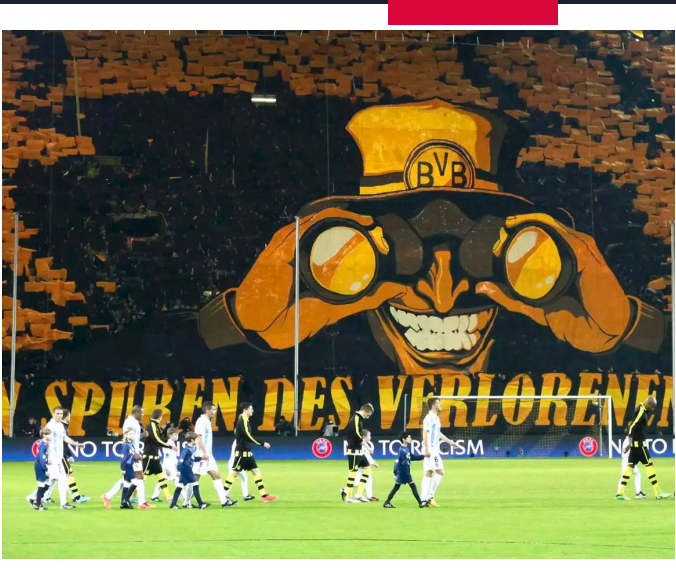
Os padrões de projeto comportamentais focam em algoritmos e atribuição de responsabilidades entre objetos, descrevendo padrões de comunicação entre eles.

Nesses padrões, encontramos uso intensivo de polimorfismo.



OBSERVER

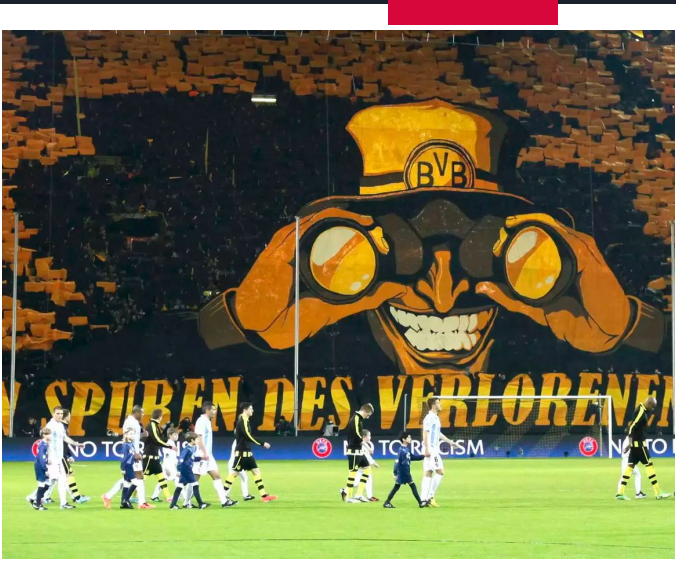




DEFINIÇÃO

Definir uma dependência um-para-muitos entre objetos, de maneira que quando um objeto muda de estado todos os seus dependentes são notificados e atualizados automaticamente.

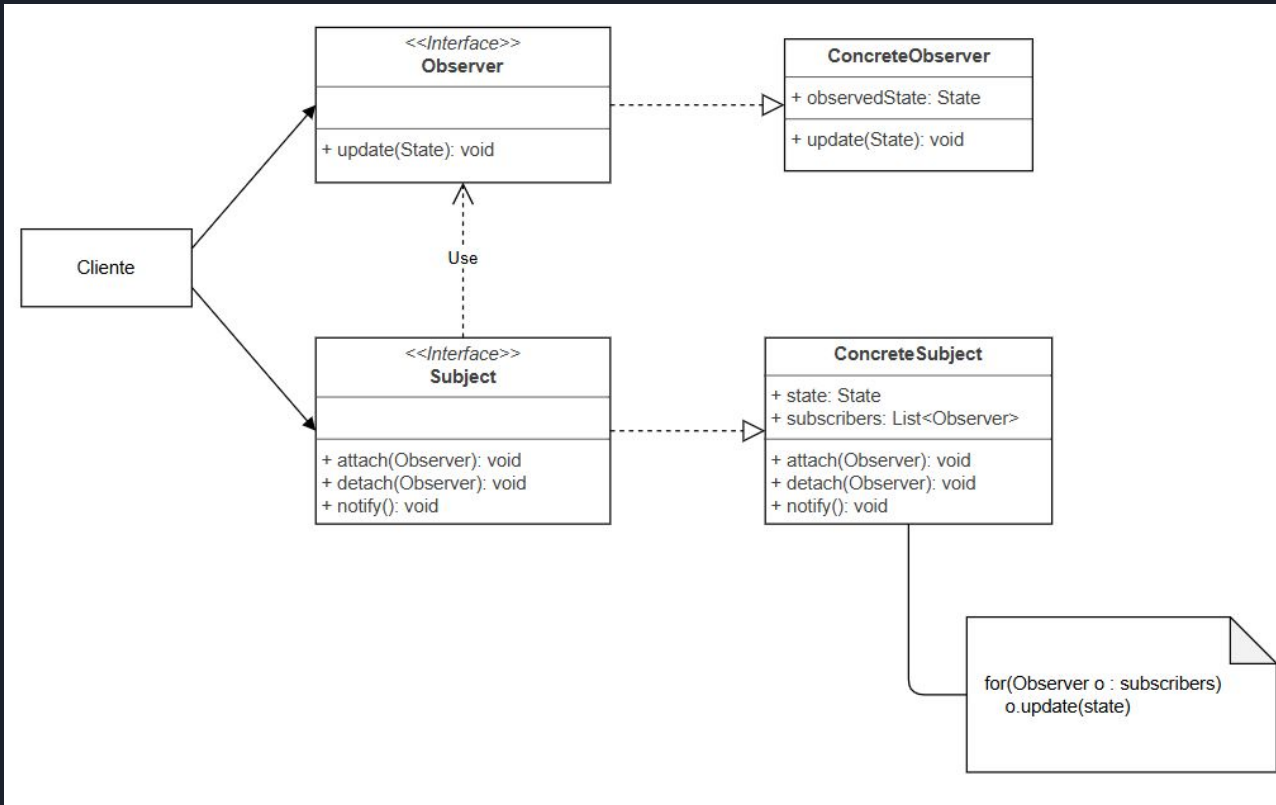
GoF, Padrões de Projetos. p.274



MOTIVAÇÃO

Imagine que você tem interesse no conteúdo de uma determinada revista, que é atualizada com periodicidade variada. Ao invés de você ficar ligando para a redação da revista para perguntar quando a nova edição será distribuída, você realiza uma assinatura e, quando uma nova edição da revista ficar pronta, ela será entregue na sua casa sem que você tenha que ficar atento.

DIAGRAMA





PARTICIPANTES

- **Subject:** interface que fornece operações para inscrever observadores e de notificar atualizações do estado gerenciado.
- **ConcreteSubject:** implementação das operações de observação do estado. Nesse ponto, todos os observadores recebem a atualização



PARTICIPANTES

- **Observer:** interface que define ponto de recebimento de atualização do estado observado
- **ConcreteObserver:** implementação da operação de recebimento do estado atualizado que está sendo observado
- **Cliente:** elemento que orquestra a utilização do estado observado, inscrevendo novos observadores



UTILIZAÇÃO

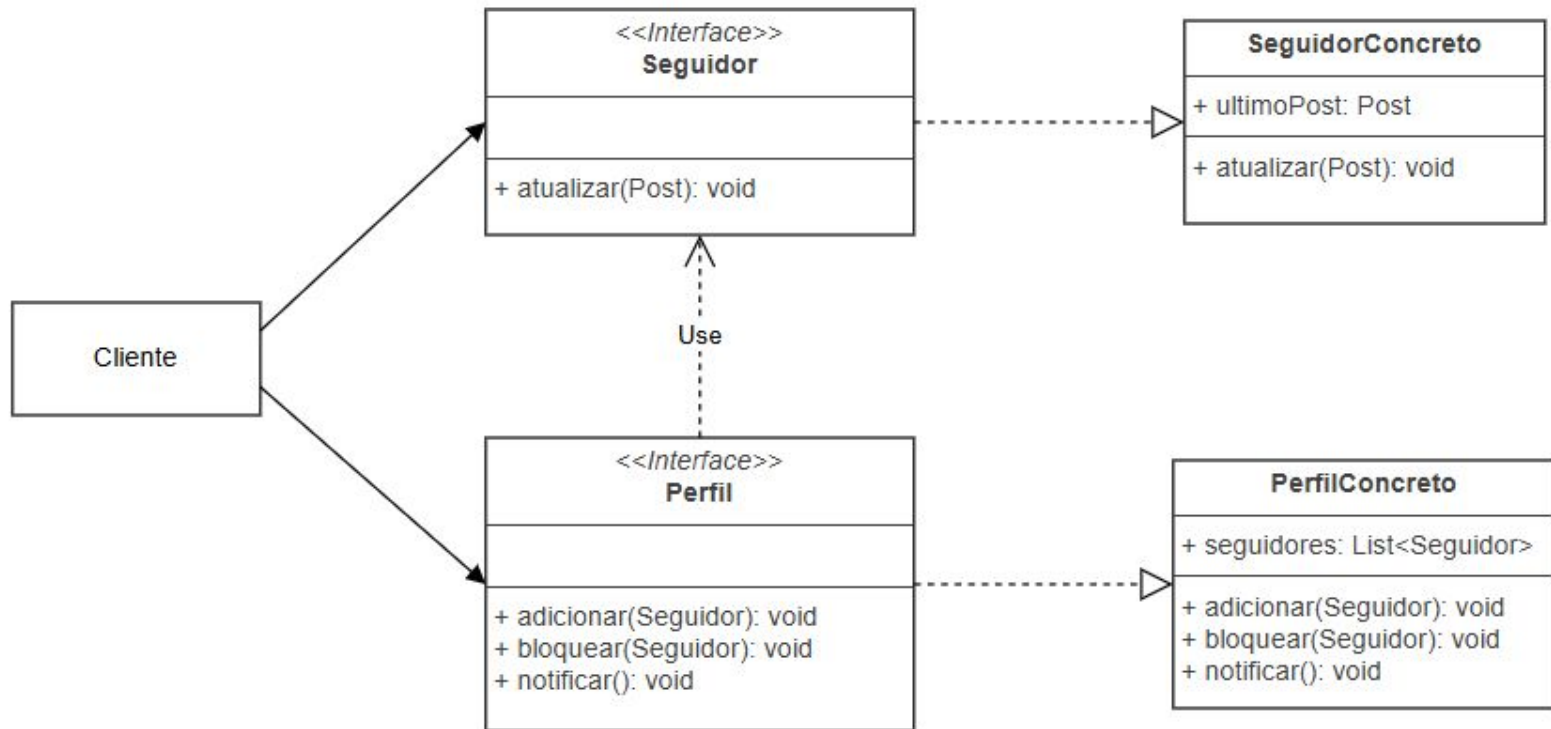
Um objeto com estado observável é criado e, a partir dele, observadores podem ser inscritos para receberem atualizações deste estado. Sempre que o estado for atualizado, o objeto percorre todos os observadores inscritos, enviado para eles o estado atualizado.



HORA DE EXERCÍCIO

Quando o Bistecones faz uma nova postagem, todos os seguidores devem ser notificados. Dessa forma, quando um novo post é publicado, o sistema percorre essa lista e chama o método “atualizar” de cada seguidor, disparando uma notificação (push, e-mail etc).

DIAGRAMA

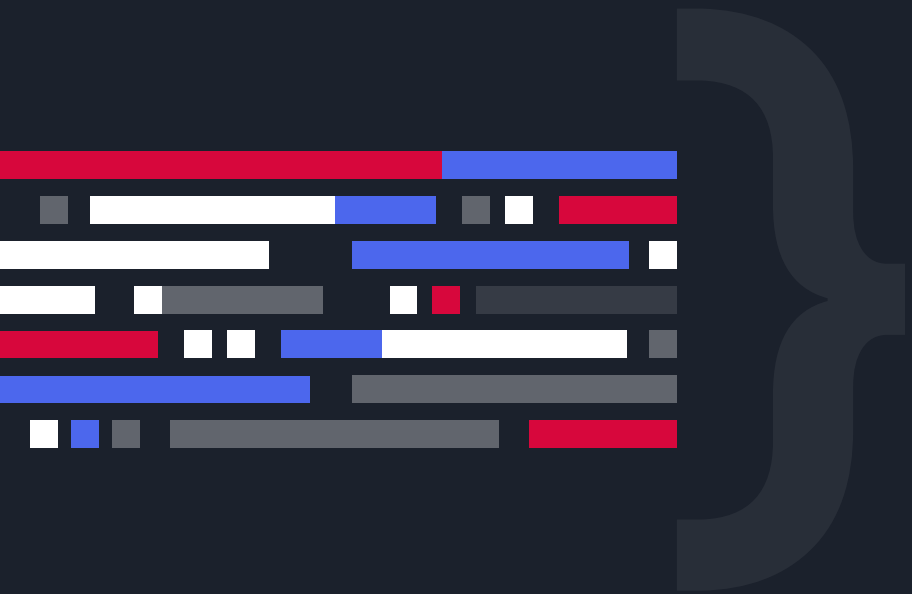




HORA DE EXERCÍCIO

Em um painel de ações ou criptomoedas, várias partes da interface (um gráfico de velas, uma tabela de preços, um histórico de transações) precisa exibir o valor atual de um ativo. Quando o preço de um ativo muda, os componentes que observam essa ação recebem o valor e atualizam a visualização na interface.

DIAGRAMA?



APLICABILIDADE

- Quando uma abstração tem dois aspectos, onde um depende do outro
- Quando a mudança em um objeto exige mudanças em outros, e você não sabe quantos objetos devem ser mudados
- Quando um objeto deveria ser capaz de notificar outros objetos sem fazer hipóteses, ou usar informações, sobre quem são esses objetos



CONSEQUÊNCIAS

- Acoplamento abstrato entre Subject e Observer
- Suporte para comunicações do tipo broadcast
- Atualizações inesperadas, sem considerar o custo global de mudança do subject



STATE





DEFINIÇÃO

Permitir a um objeto alterar seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado sua classe.

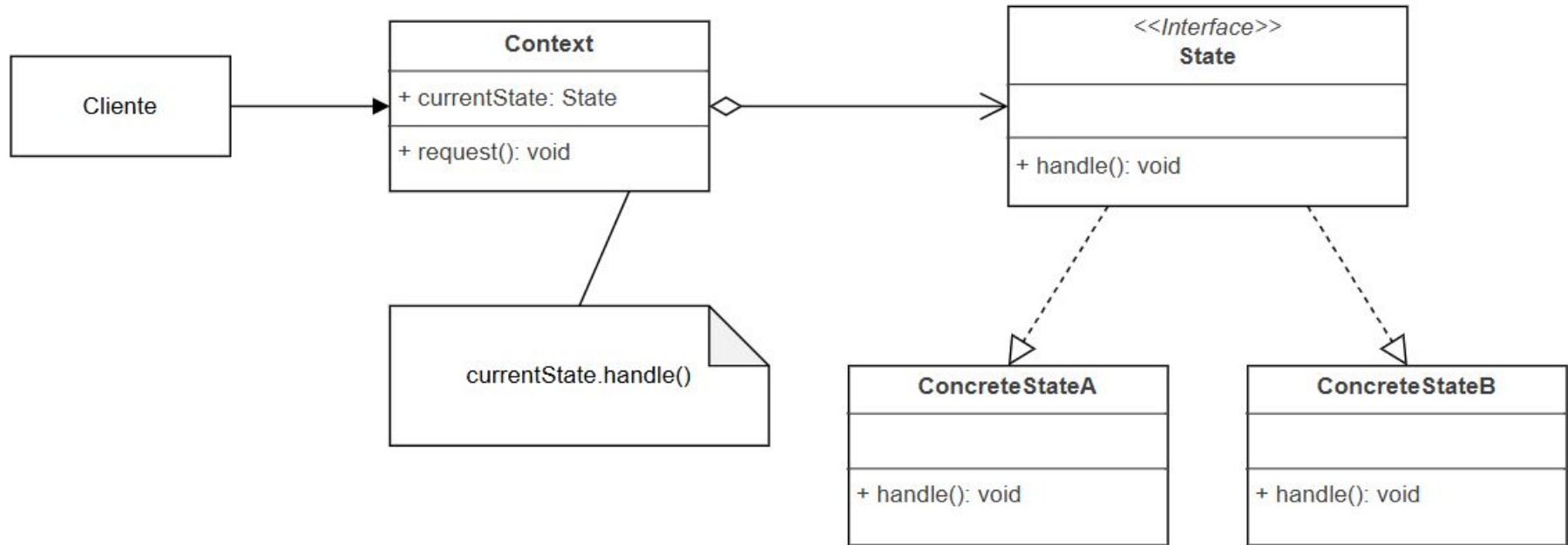
GoF, Padrões de Projetos. p.284

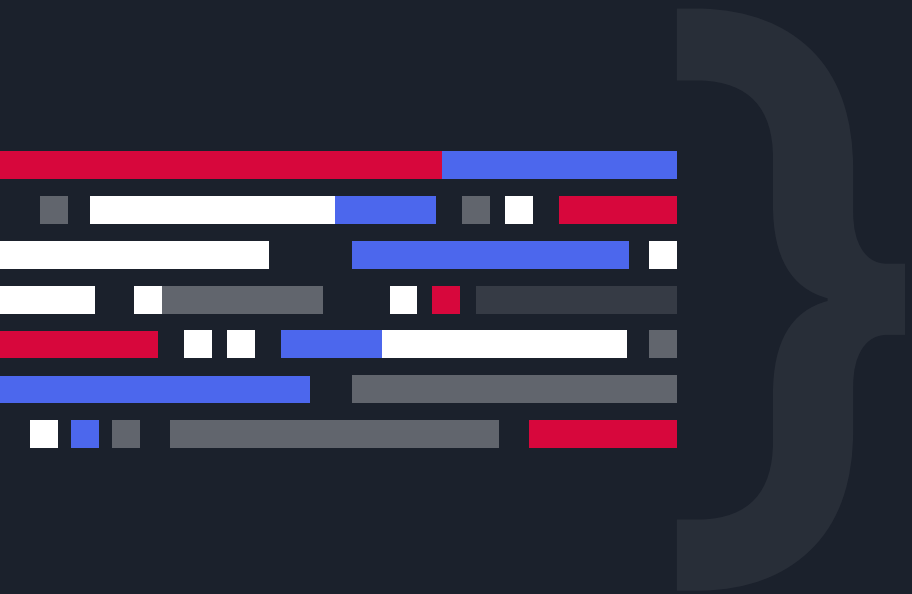


MOTIVAÇÃO

Considere uma conexão TCP em rede. Essa conexão pode assumir diferentes estados: Estabelecida, Escutando, Fechada. Cada um desses estados assume uma rotina específica, de modo que a tentativa de executar uma ação em um estado diferente não deve ser possível, em conformidade com a transição de estados que a conexão pode assumir.

DIAGRAMA





PARTICIPANTES

- **State:** interface que fornece uma operação a ser executada a partir de um estado
- **ConcreteStateX:** implementação da operação de cada estado descrito
- **Context:** classe que fornece a interface que segue a mesma interface gerenciada para cada estado
- **Cliente:** elemento que faz uso da transição de estados



UTILIZAÇÃO

Cada estado é descrito em uma implementação da interface. No contexto, ocorre a chamada do método que executa a ação de cada estado, que internamente consideram as regras específicas de transição de estado. Dessa forma, o contexto não tem a necessidade de conhecer as regras de transição para cada estado.

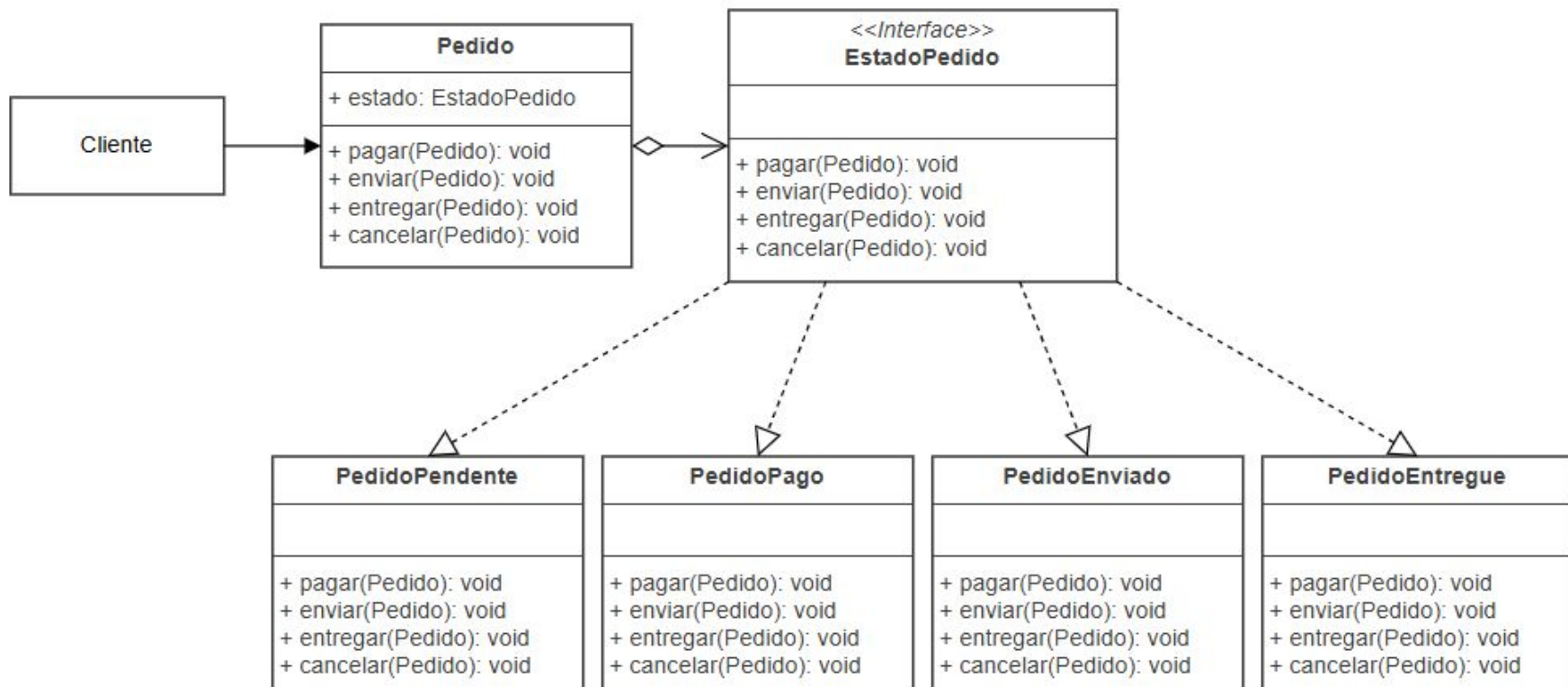


HORA DE EXERCÍCIO

Em um e-commerce, um pedido de compra passa por vários ciclos de vida, como: Pendente, Pago, Enviado, Entregue, Cancelado.

- No estado **Pendente**, você pode chamar o método "pagar()"
- No estado **Enviado**, chamar o método "pagar()" não deve fazer nada
- No estado **Entregue**, não é mais possível chamar o método "cancelar()"

DIAGRAMA

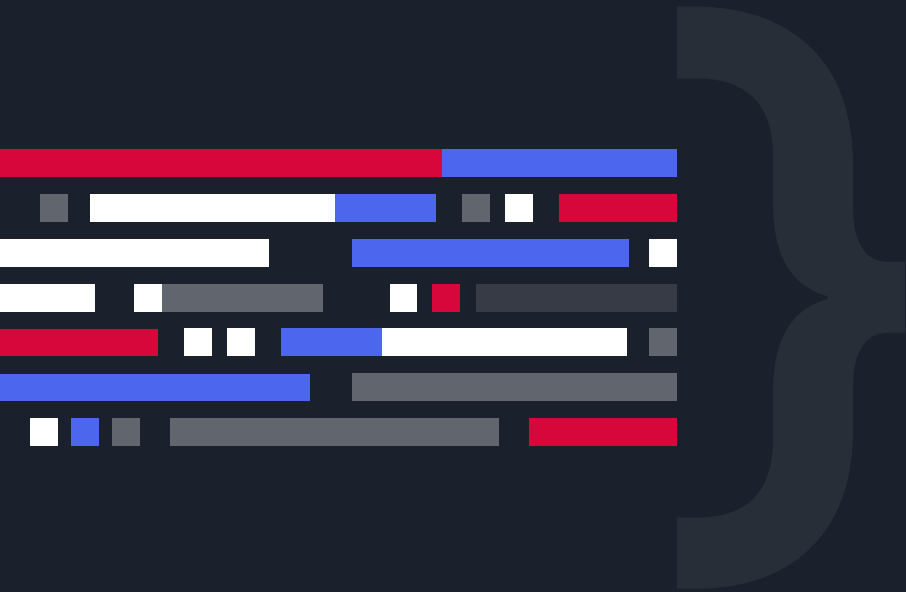




HORA DE EXERCÍCIO

Um sistema de publicação de conteúdos oferece aos redatores a possibilidade de criar artigos com uma hierarquia de aprovação antes da publicação: **Rascunho**, **Em Moderação**, **Publicado**, **Arquivado**. Enquanto o artigo estiver em **rascunho**, ele pode ser editado a vontade. Quando ele estiver em **moderação**, não poderá ser editado e apenas aprovado ou reprovado. Quando estiver **publicado**, não poderá ser editado nem aprovado, podendo apenas ser **arquivado**.

DIAGRAMA?



APLICABILIDADE

- Quando o comportamento de um objeto depende do seu estado e ele pode mudar em tempo de execução
- Quando operações têm comandos condicionais grandes, de várias alternativas possíveis, que dependem do estado do objeto



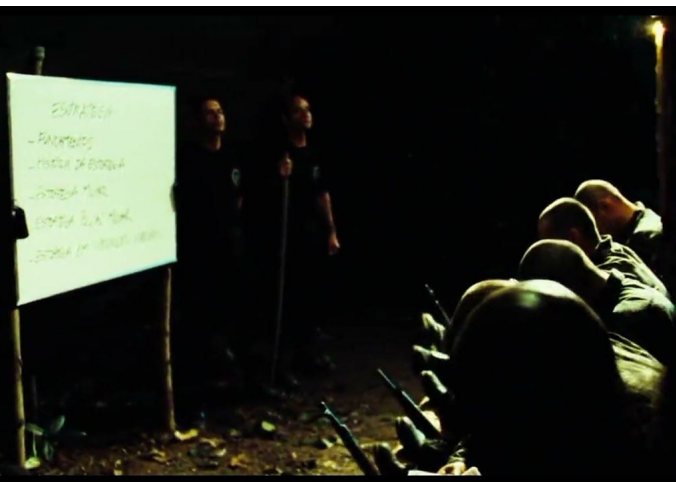
CONSEQUÊNCIAS

- Confina comportamento específico de estados e particiona o comportamento para estados diferentes
- Torna explícitas as transições de estado
- Objetos State podem ser compartilhados



STRATEGY

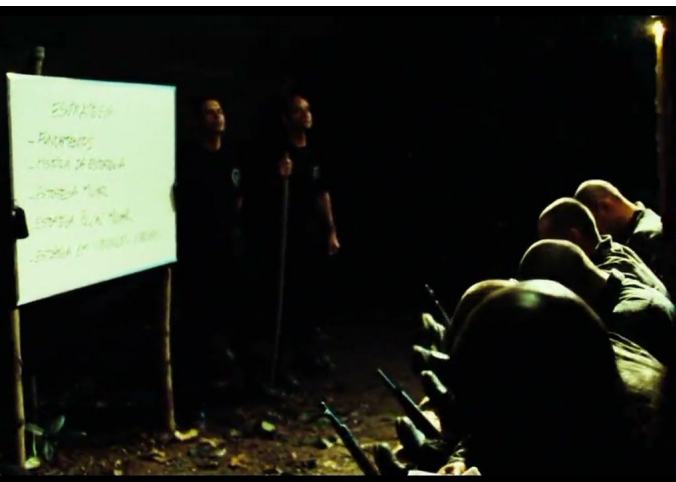




DEFINIÇÃO

Definir uma família de algoritmos, encapsular cada uma delas e torná-las intercambiáveis. Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.

GoF, Padrões de Projetos. p.292

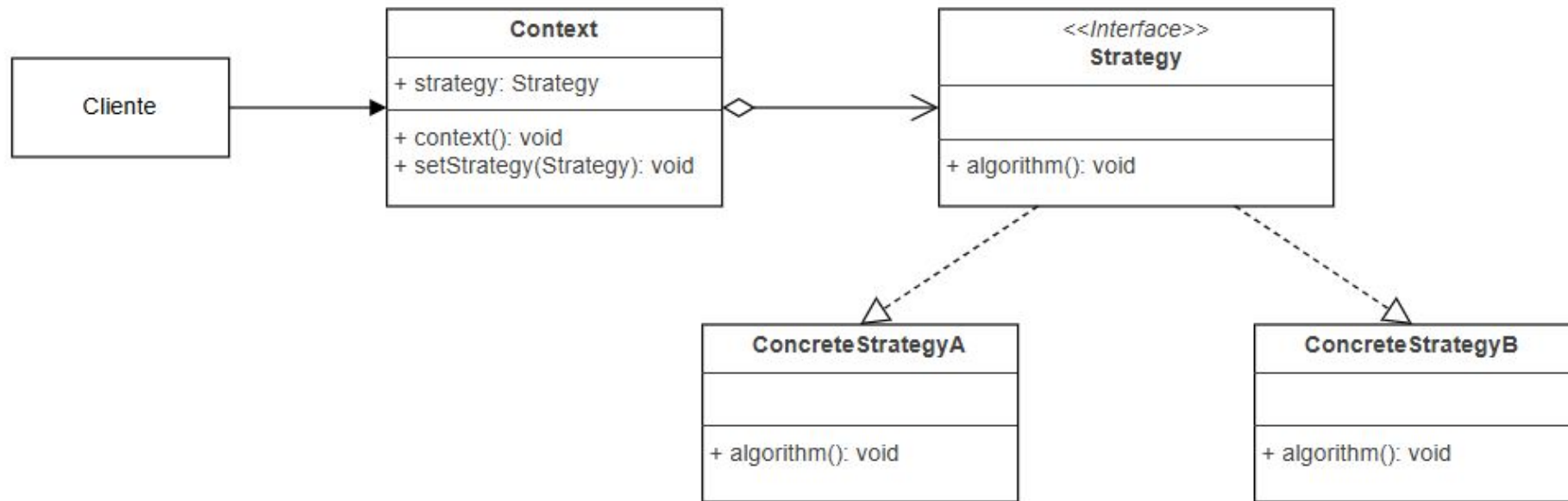


MOTIVAÇÃO

Existem muitos algoritmos para quebrar um stream de texto em linhas, mas codificar esses algoritmos de forma fixa e rígida não é tão desejável por termos menos flexibilidade para diferentes algoritmos.

Dessa forma, podemos encapsular cada um desses algoritmos em classes separadas, que formam as chamadas **estratégias**.

DIAGRAMA





PARTICIPANTES

- **Strategy:** interface que define a operação encapsulada
- **ConcreteStrategyX:** implementação da operação específica que segue uma estratégia
- **Context:** classe que faz uso de uma instância de estratégia configurada em tempo de execução
- **Cliente:** elemento que faz uso das estratégias a partir do contexto



UTILIZAÇÃO

Com base nos dados que são enviados ao contexto, ele tem a possibilidade de modificar a estratégia empregada no fluxo em tempo de execução, sem a necessidade de condições múltiplas sendo definidas no algoritmo. Isso permite uma maior flexibilidade no uso de diferentes estratégias.

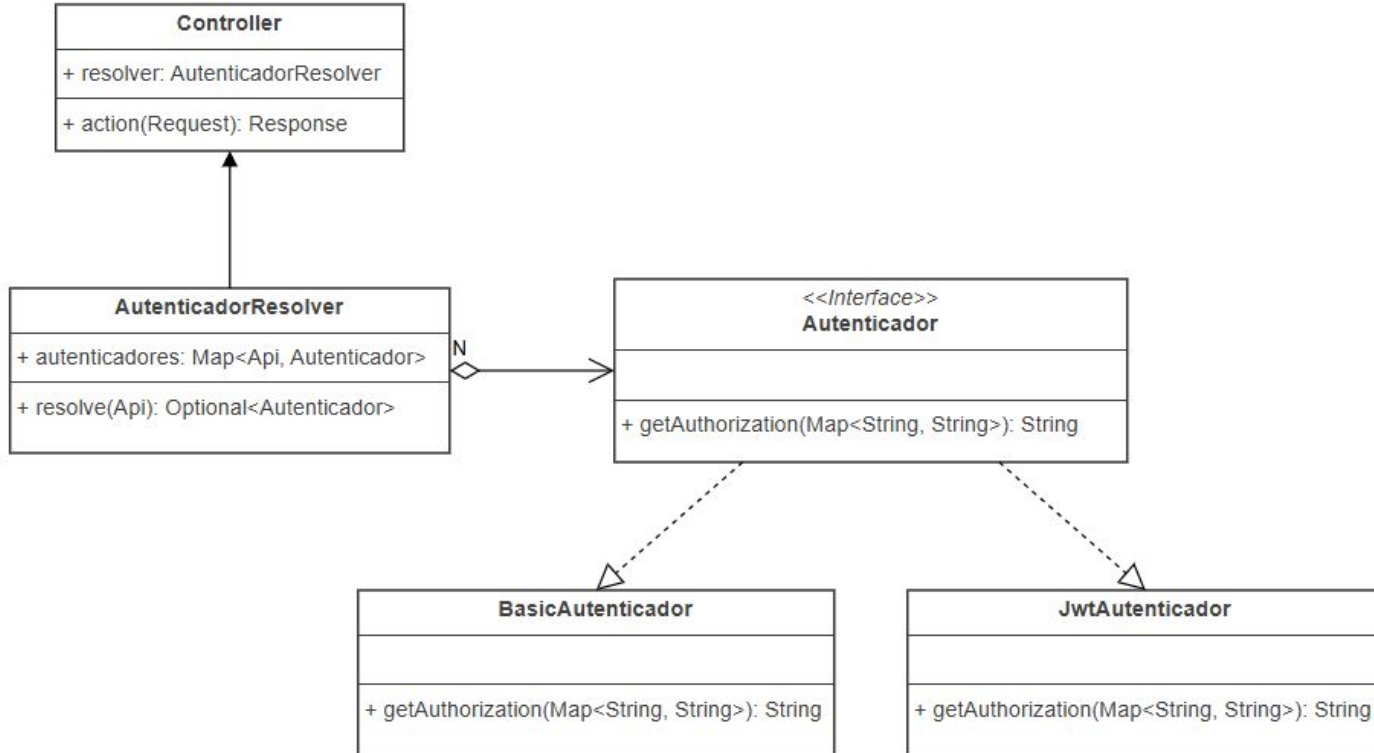


HORA DE EXERCÍCIO

Em uma integração de API, é muito comum tratarmos de mecanismos de autenticação, que vão permitir ou não que usemos as APIs. Nesse processo, é interessante que tenhamos esses mecanismos sendo configurados de forma dinâmica, se adequando ao recurso sendo solicitado.

Vamos implementar um mecanismo de gerenciamento de autenticação de APIs.

DIAGRAMA



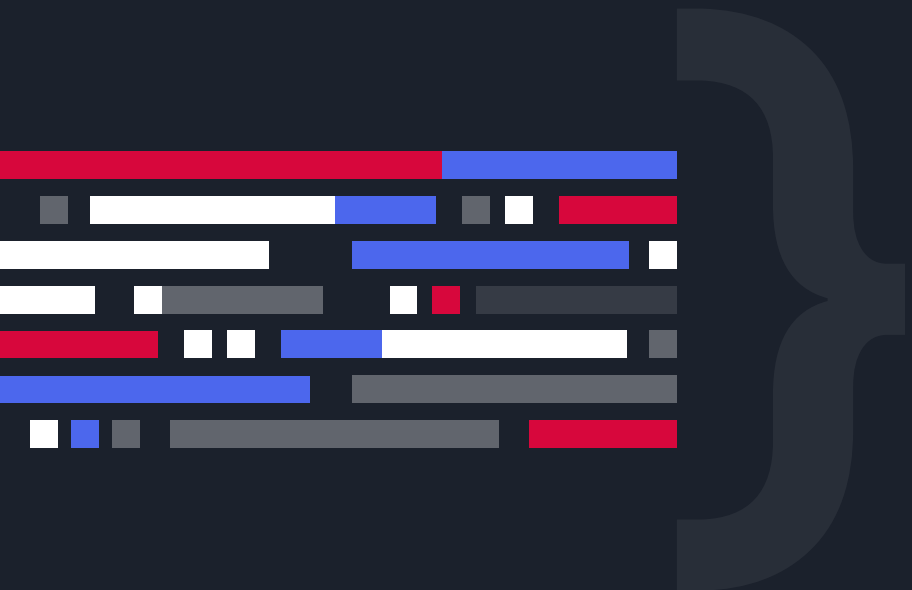


HORA DE EXERCÍCIO

Na hora de concluir uma compra em um sistema de e-commerce, podemos decidir qual forma de pagamento iremos utilizar. O carrinho de compras não precisa conhecer a fundo detalhes de formas de pagamento (PIX, cartão, boleto etc), mas sim saber que, chamando um método “pagar(valor)” essa operação vai ser executada e ser aprovada ou não.

Vamos implementar esse sistema.

DIAGRAMA?



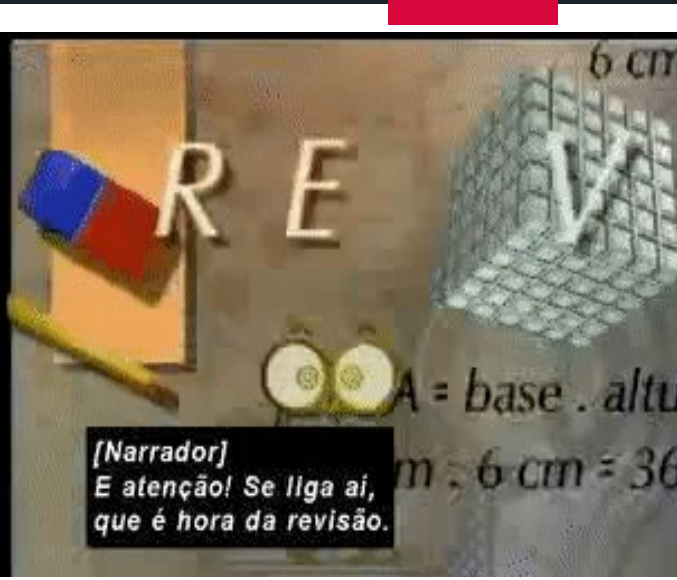
APLICABILIDADE

- Quando muitas classes relacionadas diferem somente no seu comportamento
- Quando há a necessidade de termos variantes de um algoritmo
- Quando uma classe define muitos comportamentos



CONSEQUÊNCIAS

- Famílias de algoritmos relacionados
- Ter uma alternativa ao uso de subclasses
- Estratégias eliminam comandos condicionais da linguagem de programação
- Possibilidade de escolha de implementações
- Custo de comunicação e aumento no número de objetos



RECAPITULANDO

- Padrões de projeto comportamentais
- Observer
- State
- Strategy

OBRIGADO!



"ACOOOOORDA MENIIIIINA!"

Ana Maria Braga